Laptop Rentals

AlgoExpert · Hard · Heaps

The problem

- Given [start, end) rental intervals, minimum number of laptops needed so nobody waits.

Intuition

- The number of laptops needed equals the maximum number of overlapping rentals.

Approach

- 1. Sort starts and ends separately
- 2. Sweep with two pointers
- 3. A start before the next end needs a laptop
- 4. Otherwise reuse one

Java solution

```java
public class LaptopRentals {
    public static int laptopRentals(int[][] times) {
        int[][] t = times.clone();
        Arrays.sort(t, (a, b) -> Integer.compare(a[0], b[0]));
        PriorityQueue<Integer> ends = new PriorityQueue<>();
        for (int[] iv : t) {
            if (!ends.isEmpty() && ends.peek() <= iv[0]) ends.poll();
            ends.add(iv[1]);
        }
        return ends.size();
    }
}
```

Complexity

- Time: O(n log n)
- Space: O(n)

Recap

- The number of laptops needed equals the maximum number of overlapping rentals.
- Complexity: O(n log n)